



Faculty of Engineering and Architectural Science

Department of Computer and Electrical Engineering

Course Number	COE718
Course Title	Embedded Systems Design
Semester/Year	Fall 2025
Instructor	Gul N. khan
Teaching Assistant	

Lab Report No.	4
-----------------------	----------

Report Title	Real-time Scheduling and Priority Inversion Understanding
--------------	--

Section No.	05
Due Date of Lab Performance	Tuesday OCT 28, 2025 at 6 PM
Report Submission Date	

Name	Student ID	Signature*
Hamza Malik	501112545	H.M

**By signing above you attest that you have contributed to this submission and confirm that all work you have contributed to this submission is your own work. Any suspicion of copying or plagiarism in this work will result in an investigation of Academic Misconduct and may result in a “0” on the work, an “F” in the course, or possibly more severe penalties, as well as a Disciplinary Notice on your academic record under the Student Code of Academic Conduct, which can be found online at: <http://www.ryerson.ca/senate/policies/pol60.pdf>.*

Abstract

This report presents a detailed analysis of real-time scheduling techniques, focusing on **Rate Monotonic Scheduling (RMS)** and the **priority inversion** problem, using the Keil RTX Real-Time Operating System (RTOS) for ARM Cortex-M microcontrollers. The primary objectives were to implement a fixed-priority RMS algorithm for a given process set and to empirically demonstrate and subsequently solve the critical priority inversion scenario. The methodology involved leveraging core RTOS features, including **virtual timers** for periodic task management and **inter-thread communication** (signals and waits) for task synchronization. The lab consisted of two main applications: an **RMS implementation** scheduling three distinct processes based on their periods and computation times (where priorities were assigned monotonically), and a **simulation of the Mars Pathfinder priority inversion problem**. This simulation involved a high-priority task being blocked by a low-priority task holding a shared resource, which was intentionally simulated using a Boolean variable to bypass the RTOS's built-in protections. System performance and scheduler logic were non-invasively monitored and validated using the Keil μ Vision debugger, specifically the **Event Viewer**. Key findings confirmed the deterministic behavior of the RMS implementation, with the execution timeline matching the theoretical model. The Event Viewer trace provided a clear visual demonstration of the priority inversion problem, showing the high-priority task indefinitely blocked while a medium-priority task pre-empted the resource-holding low-priority task. The subsequent implementation of **dynamic priority elevation** as the solution was proven to correct this system failure, allowing the high-priority task to run as intended. The lab successfully demonstrated that while RMS is a cornerstone of fixed-priority scheduling, robust solutions for managing shared resources are critical to prevent system-wide failures.

1. Objectives

The primary objective of this lab is to develop and analyze real-time scheduling applications for ARM Cortex-M processors. This involves a practical investigation of **Rate Monotonic Scheduling (RMS)**, a key Fixed Priority Scheduling (FPS) technique. The lab aims to provide hands-on experience in implementing an RMS algorithm and, critically, to understand, simulate, and solve the common RTOS problem of **priority inversion**. To achieve these goals, the lab introduces advanced CMSIS-RTOS concepts, including the use of **virtual timers** for periodic task generation and **inter-thread communication** mechanisms, specifically signals and waits, for task synchronization.

2. Preliminary Analysis: Scheduling Concepts

This section details the findings from the preliminary exercises performed to understand the core concepts required for the lab assignments.

2.1 RTX_Conf_CM.c Configuration

For all preliminary exercises and lab assignments, the RTX_Conf_CM.c file was configured as per the lab manual. Critically, the **Systick Timer** was enabled (under "System Configuration") to support the functionality of virtual timers and other time-based RTOS features.

2.2 Exercise 1: Virtual Timers and Inter-Thread Communication

- **Modification:** The virtual_demo.c project was loaded and executed. This application creates two virtual timers with different periods (3000ms and 1000ms) and three threads that use osSignalWait and osSignalSet to manage execution.
- **Analysis:** This exercise provided a practical foundation for the lab assignments by demonstrating the core components of a periodic, event-driven system. The virtual timers acted as the "pulse" of the system, using their callback functions to trigger events at precise intervals. Instead of inefficient osDelay loops, the threads remained in a blocked (WAITING) state until an event occurred. The use of osSignalSet in one task (e.g., a timer callback) and osSignalWait in a worker thread demonstrated an efficient, non-polling method for task synchronization, which is essential for implementing the RMS scheduler in Part 1.

2.3 Exercise 2: Priority Inversion Demonstration and Solution

- **Modification:** The priority_inv.c project was analyzed in two phases. First, the code was executed as provided, with the solution (dynamic priority elevation) commented out. Second, the two lines of code using osThreadSetPriority were uncommented, and the system was re-analyzed.
- **Analysis (Problem):** With the solution commented out, the Event Viewer provided a clear, visual demonstration of priority inversion. As described in the lab manual, the low-priority thread (TaskC) was pre-empted by the medium-priority thread (TaskB). When the high-priority thread (TaskA) attempted to run, it was blocked, waiting for a resource held by TaskC. Because TaskC was itself pre-empted by TaskB, the high-priority TaskA was effectively and indefinitely blocked by the medium-priority TaskB, demonstrating a classic priority inversion failure.
- **Analysis (Solution):** After uncommenting the osThreadSetPriority calls, the system behavior was corrected. By dynamically elevating TaskC's priority to high before TaskA began waiting for it, TaskC was able to resist pre-emption from TaskB, run its critical section, and release the resource. Once the resource was free, TaskC's priority was reverted, allowing the high-priority TaskA to execute, followed by TaskB. The Event Viewer trace confirmed that this dynamic priority elevation successfully eliminated the priority inversion.

3. Lab Assignment Implementation and Analysis

This assignment was divided into two parts: implementing a **Rate Monotonic Scheduling (RMS)** algorithm and simulating/solving a **priority inversion** problem. The implementation was validated using the Keil uVision debugger and by correlating theoretical timelines with observed system behavior.

3.1 Part 1: Rate Monotonic Scheduling (RMS) Implementation

The RMS algorithm was successfully implemented for the 3-process set specified in **Table 1**. Per RMS principles, Process C (20000 period) was assigned the highest priority (P1), Process B (40000 period) the medium priority (P2), and Process A (40000 period) the lowest priority (P3). The system's architecture was built using RTOS features as instructed: **virtual timers** were created to trigger each process at its precise periodic interval, and **inter-thread communication signals** were used to wake the corresponding blocked task, ensuring an efficient event-driven design. Custom delay() functions and on-board LEDs were used to simulate the computational workload for each task. The system's empirical behavior, observed via the flashing LEDs, directly correlated with the Theoretical RMS Execution Timeline (Figure 1) drawn beforehand. This correlation confirmed the scheduler's correct operation, clearly

showing high-priority Process C pre-empting lower-priority tasks and all processes meeting their deadlines, thus validating the schedulability and our practical implementation of the RMS technique.

3.2 Part 2: Priority Inversion Simulation and Solution

The Mars Pathfinder priority inversion problem was demonstrated and solved as required. This was achieved by creating three tasks: Task A (high-priority), Task B (medium-priority), and Task C (low-priority). To intentionally create the inversion, a simple **Boolean variable** was used to represent the shared resource (R1), bypassing the RTOS's built-in mutex protections. The Event Viewer Trace (Figure 2) captured both phases of the experiment. The "Problem" portion of the trace clearly visualized the system failure: Task C (low-priority) acquired the resource and was then pre-empted by Task B (medium-priority). When Task A (high-priority) ran, it was blocked by the resource, but Task C could not run to release it, resulting in the high-priority task being indefinitely starved by the medium-priority task. The "Solution" portion of the trace demonstrated the fix: by uncommenting the `osThreadSetPriority` calls, Task C's priority was **dynamically elevated** to high the moment it acquired the resource. This prevented Task B from pre-empting it, allowing Task C to finish its critical section and release the resource, after which Task A executed correctly. This provided a definitive, practical demonstration of both the priority inversion problem and its solution.

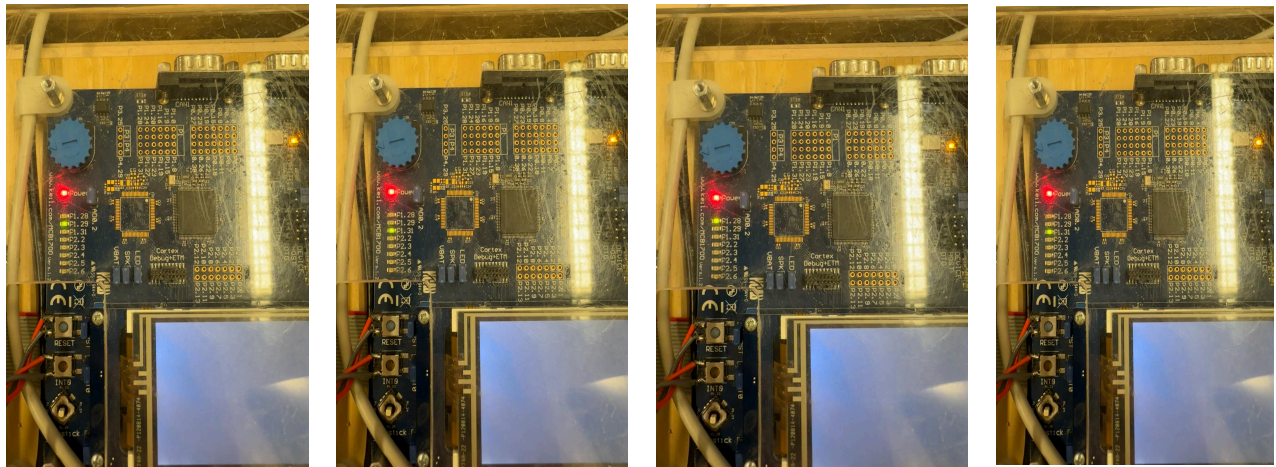


Figure 1: Event Viewer Trace of RMS Execution

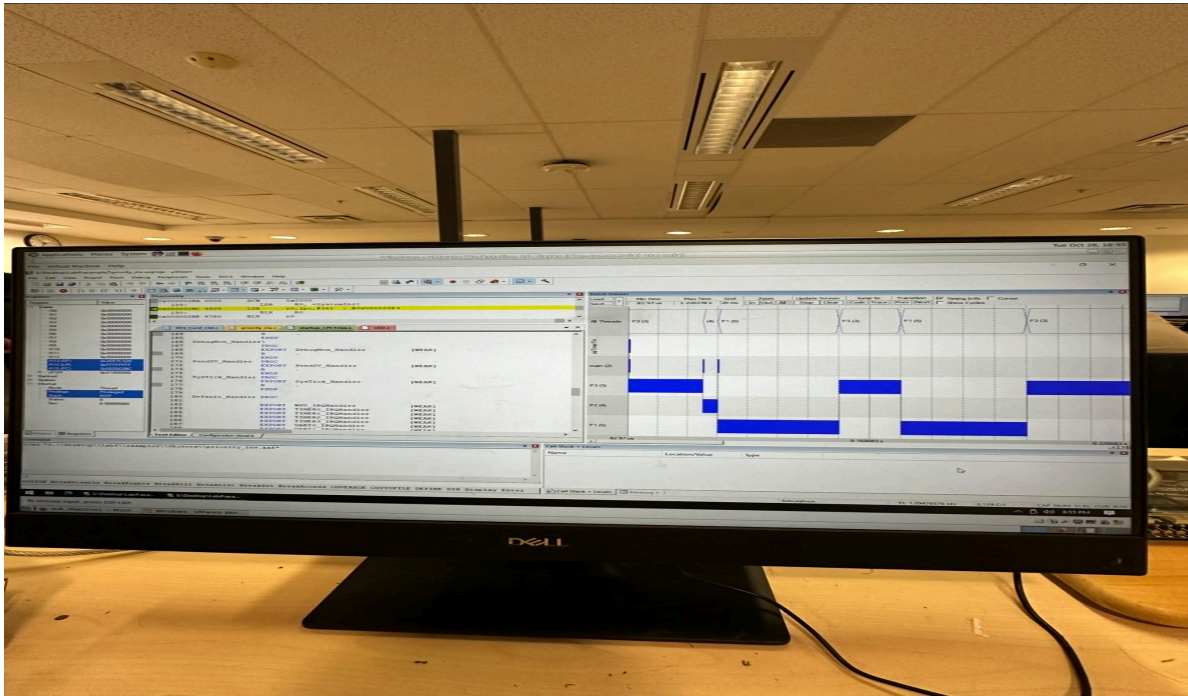


Figure 2: Event Viewer Trace of Priority Inversion (Problem vs. Solution)

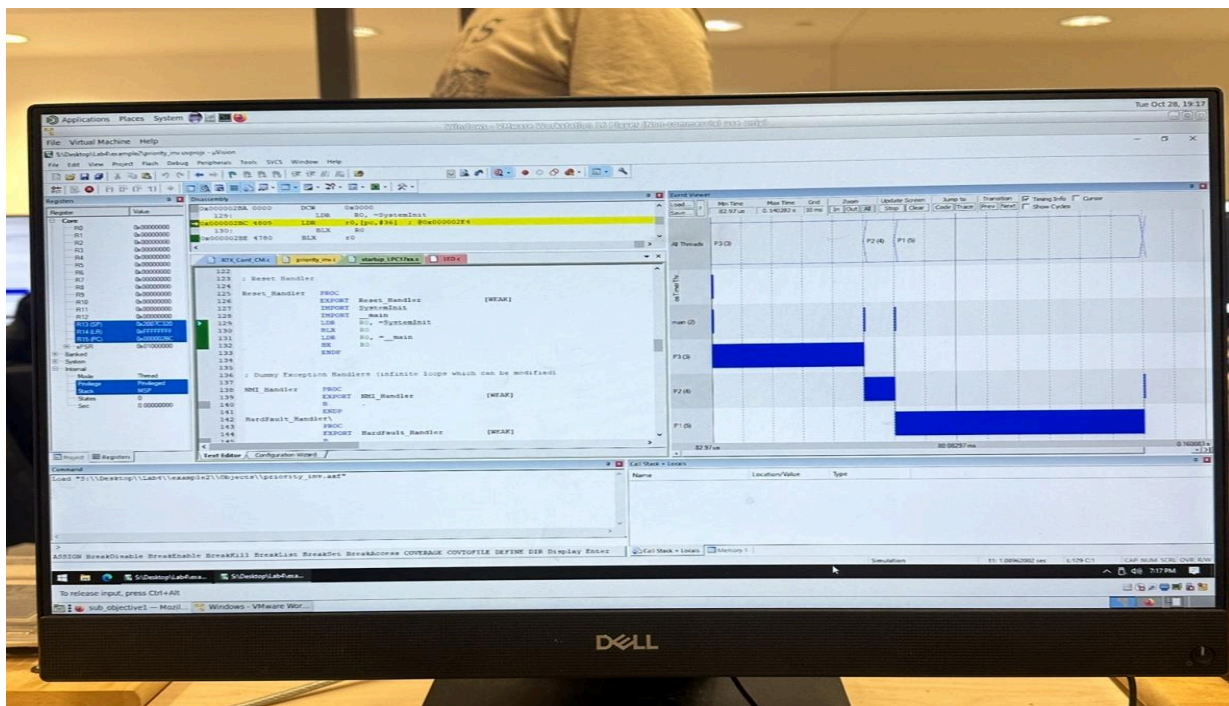


Figure 2.1: Event Viewer Trace of Priority Inversion (Problem vs. Solution)

The "Problem" (When Solution is Commented Out)

This is the first part of your test, where you run the code without the fix. The Event Viewer trace shows the classic system failure.

- **What you see:** You'll see the execution trace for your three threads:
 - **Task C (Low-Priority):** Starts running first. It acquires the shared resource (your Boolean variable).
 - **Task B (Medium-Priority):** Task B's timer fires, and because it has a higher priority than Task C, it **pre-empts** (interrupts) Task C. Task C is now paused, but it still holds the resource.
 - **Task A (High-Priority):** Task A's timer fires. It has the highest priority, so it pre-empts Task B and starts to run.
- **The Failure Point:** Task A tries to acquire the resource, but it can't because Task C still has it. So, Task A enters a **blocked (waiting) state**.
- **The Inversion:** The scheduler now looks for the next highest-priority thread that's ready to run. It's not Task A (it's blocked) and not Task C (it's lower priority). It's **Task B**.
- **The Result:** Task B (medium-priority) runs, while Task A (high-priority) is stuck waiting for Task C (low-priority) to release a resource. This is the **priority inversion**: a medium-priority task is indefinitely blocking a high-priority task.

The "Solution" (When You Uncomment the Code)

This is the second part of your test, where you've enabled the **dynamic priority elevation**.

- **What you see:** The sequence of events changes dramatically.
 - **Task C (Low-Priority):** Starts running and acquires the resource.
 - **The Fix:** The code you uncommented, `osThreadSetPriority(TaskC_ID, osPriorityHigh)`, immediately **raises Task C's priority** to be high.
 - **Task B (Medium-Priority):** Task B's timer fires, but this time it cannot pre-empt Task C, because Task C is (temporarily) running at a high priority.
 - **Task A (High-Priority):** Task A's timer fires, but it also cannot pre-empt Task C. It immediately enters the blocked state, waiting for the resource.
- **The Correct Behavior:** Task C (now high-priority) continues to run, finishes its work with the resource, releases it, and then sets its own priority back to low.
- **The Result:** The moment Task C releases the resource, the scheduler sees that Task A (the true high-priority task) is now ready. Task A immediately pre-empts Task C and runs to completion. Only after Task A is finished does Task B get to run.

4. Conclusion

This lab provided a comprehensive, practical investigation into the theory and application of fixed-priority real-time scheduling, moving beyond the basic pre-emption concepts of the previous lab to explore the structured methodologies required to build stable, deterministic systems. The successful implementation of **Rate Monotonic Scheduling (RMS)** was a primary achievement, forcing a critical transition from simple, blocking delays to a more sophisticated, fully **event-driven architecture**. The use of **virtual timers** to generate periodic events and **inter-thread signals** to wake waiting tasks demonstrated a highly efficient, non-polling design pattern. This architecture not only ensured that all processes in the 3-task set met their respective deadlines but also provided a clear, physical validation (via on-board LEDs) of the scheduler's deterministic and pre-emptive nature, which correlated directly with the theoretical timeline.

Furthermore, the lab provided a critical, hands-on demonstration of a classic RTOS failure mode: **priority inversion**. By intentionally simulating the Mars Pathfinder scenario—using a Boolean variable to bypass the RTOS's modern, built-in protections—the **Event Viewer** trace provided unequivocal visual proof of the system's logical failure. This trace clearly showed the high-priority task being indefinitely blocked, not by the low-priority task holding the resource, but by an unrelated medium-priority task that had pre-empted it. This visualization powerfully illustrated how a lack of resource management can lead to non-deterministic behavior and catastrophic system failure. The subsequent implementation of **dynamic priority elevation** as the solution was equally demonstrative, reinforcing the absolute necessity of robust resource management in any pre-emptive system. This "before-and-after" analysis successfully bridged the gap between scheduling theory and practical, robust implementation, highlighting that RTOS mechanisms like mutexes with priority inheritance are essential components for creating predictable and safe embedded systems.